Pointwise Worst-Case Resource Complexity: From Partial-Information Sorting to Public-Instance Optimality

Yunbei Xu
National University of Singapore
yunbei@nus.edu.sg

Abstract

Worst-case complexity can hide the structure legitimately revealed by an input. This note formalizes pointwise worst-case resource complexity for revealed-instance problems: a public instance is fixed, an adversary chooses a compatible hidden completion, and the resource may be any model-specific measure, such as comparisons, running time, space, communication, or expected randomized cost. Sorting under partial information is the clean finite case. Given a public partial order P, the hidden completions are its linear extensions, and the optimal worst-case number of additional comparisons is $\Theta(\log_2 e(P))$, where $e(P)$ is the number of compatible extensions.

Existing partial-sorting results show that one uniform comparison algorithm matches, for every P, the clairvoyant benchmark that is allowed to know P but not the hidden extension. We interpret the comparison upper bound through an explicit Bellman log potential. We then extend the same sorting example beyond finite inputs by revealing finite-resolution bucket labels of real keys; the residual pointwise cost is $\Theta(\sum_a \log_2(m_a!))$. The formulation also aligns pointwise universal optimality with two established precedents in theoretical computer science: order-oblivious instance-optimal geometry, which is pointwise in the unordered point set after quotienting over input permutations, and universal optimality for graph algorithms, which is pointwise in the fixed graph topology under adversarial weights. The surrounding Occam, finite-scale statistical-dimension, and polyhedral-volume readings are kept typed: they illuminate the same logarithmic quantity, but the theorem here remains a deterministic resource statement.

1 Introduction

Worst-case analysis asks for one number. For comparison sorting on n items, the answer is $\Theta(n \log n)$. The answer is correct, but it is too compressed: it gives the same label to an entirely unordered list and to a list whose order is almost already known.

Sorting with partial information is the simplest place where this compression can be undone. The algorithm is given a partial order P on the items. The hidden truth is an arbitrary total order extending P. The algorithm may ask additional comparisons and must recover the hidden total order. If

$$e(P) := |\operatorname{Lin}(P)|$$

denotes the number of linear extensions of P, then $\log_2 e(P)$ is the number of remaining binary decisions. The theorem says that this is not only an information lower bound. It is the correct pointwise comparison complexity, up to a universal constant.

This note is precise about what is new. The comparison lower bound is the classical decision-tree counting bound. The asymptotically matching upper bound belongs to the long theory of sorting under partial information (Fredman, 1976; Kahn and Saks, 1984; Kahn and Kim, 1995; Cardinal

et al., 2013; Haeupler et al., 2025; Rutschmann, 2026). Technically, this note is primarily a faithful reformulation of sorting under partial information. Its additional contribution is threefold:

- to formulate sorting under partial information as an exact example of a general benchmark for pointwise worst-case resource complexity;
- to interpret comparison complexity in the sorting resource model through a simple Bellman potential viewpoint;
- to develop a nontrivial deterministic comparison application over uncountable key spaces through its connection with pointwise dimension in learning.

In the partial sorting example, the public point is P; the hidden states are $\mathrm{Lin}(P)$; the resource is the number of additional comparisons; and the pointwise comparison complexity is

$$d_{\mathrm{sort}}(P) = \log_2 |\mathrm{Lin}(P)|.$$

The broader lesson is about pointwise universal optimality and information revelation. If the information revealed to the learner or algorithm is formulated as the instance, then one can take a worst case over the hidden completions that remain. The benchmark may be clairvoyant about the public instance P: it may choose the best decision tree after seeing P, but it still must work for every hidden extension in $\mathrm{Lin}(P)$. The theorem below shows universal instance-optimality in this sense. A single globally valid comparison rule matches, for every P, the same order of comparisons as this instance-wise clairvoyant benchmark.

This positioning also explains why the framework is not special to sorting. In the universal-optimality theorem for Dijkstra's algorithm, the public instance is a graph topology together with a source, while the hidden completions are the admissible edge-weight assignments. The pointwise cost of an algorithm is its worst-case resource over weights for that fixed topology, and universal optimality means that one algorithm matches the topology-wise optimum for every topology (Haeupler et al., 2024). Sorting under partial information is used here because the same quantifiers are visible in their simplest exact form: public P, hidden $\mathrm{Lin}(P)$, and comparison resource.

The central proof mechanism is a logarithmic Bellman potential. We use "Bellman" in the elementary dynamic-programming sense of a resource-to-go function: after each admissible answer, the potential must fall by at least the cost of the query. This connects the note to the Bellman-sufficient viewpoint on complexity (Xu, 2026b), but here no abstract existence principle is invoked; the potential is written down explicitly. A theorem of Kahn and Saks gives a universal constant $\beta < 1$ such that every non-total poset admits an incomparable pair whose two possible comparison outcomes each leave at most a β-fraction of the linear extensions. With their constant one may take $\beta = 8/11$. Therefore

$$\Phi(P) := \frac{\log_2 e(P)}{\log_2(11/8)}$$

drops by at least one after the chosen comparison, no matter which answer the hidden extension gives. This proves the universal comparison upper bound. It is not a computationally efficient way to find the pair, because exact extension counts are hard; efficient implementations use deeper algorithms. But for comparison complexity, the potential is explicit and the quantifiers are transparent.

The finite theorem also points beyond finite state spaces. If the keys are real numbers in $[0, 1)$, an exact input has uncountably many possible values. But a practical sorting system often first sees a finite-resolution observation, such as the first b bits of each key. Those bits create ordered buckets.

If bucket a contains m_a keys, then the remaining exact sorting cost is

$$\Theta\left(\sum_a \log_2(m_a!)\right).$$

This is the sorting analogue of the finite-scale move in pointwise statistical dimension (Li and Xu, 2026): the relevant object is not the singleton real input, but the finite-resolution cell that remains after coarse information has been revealed. The analogy is about localization and scale, not about the resource being measured; here the resource is still additional comparisons.

2 Revealed-Instance Resource Complexity

The point of the framework is not to privilege running time. Following the lesson of Blum's axiomatic and machine-independent treatment of complexity (Blum, 1967), a complexity statement must specify the computational model and the resource being measured. The resource may be comparison queries, RAM steps, pointer-machine operations, communication rounds, memory, or an expected cost for a randomized algorithm. Once the resource is fixed, the pointwise cost function records how that resource varies with the revealed instance.

Definition 2.1 (Revealed-instance resource problem)**.** *Fix an input size parameter n. A revealed-instance resource problem consists of the following data.*

(i) A set $\mathcal{X}_n$ of public instances. This is the information legitimately available to the algorithm before the hidden completion is known.

(ii) A hidden-completion universe Ω_n and a relation

$$\mathcal{H}_n \subseteq \mathcal{X}_n \times \Omega_n.$$

For $x \in \mathcal{X}_n$, the compatible hidden completions are

$$\Omega_n(x) := \{\omega \in \Omega_n : (x, \omega) \in \mathcal{H}_n\}.$$

We assume $\Omega_n(x) \neq \varnothing$ for every public instance under consideration. This equation is the definition of the previously informal hidden completion space.

(iii) An output set $\mathcal{Y}_n$ and a correctness relation

$$\Gamma_n \subseteq \mathcal{X}_n \times \Omega_n \times \mathcal{Y}_n.$$

An output y is correct on (x, ω) when $(x, \omega, y) \in \Gamma_n$.

(iv) A class $\mathfrak{A}_n$ of globally admissible algorithms. An algorithm $A \in \mathfrak{A}_n$ receives the public instance x, interacts with the hidden completion only through the operations allowed by the model, and returns an output $\mathrm{Out}_A(x, \omega)$. It is globally correct if

$$(x, \omega, \mathrm{Out}_A(x, \omega)) \in \Gamma_n \quad \text{for every } x \in \mathcal{X}_n \text{ and every } \omega \in \Omega_n(x).$$

(v) A resource functional

$$\mathcal{R}_n(A; x, \omega) \in [0, \infty]$$

defined for executions of algorithms on compatible pairs. The symbol $\mathcal{R}$ is typed by the model. In a comparison model it may count only comparison queries; in a RAM model it may count total steps; in a distributed model it may count rounds; for randomized algorithms it may denote expected resource after fixing (x, ω), or another explicitly stated risk functional. Theorems must state which resource is being measured.

The analogy with Blum's machine-independent axiomatic theory of complexity is modest, but important. For ordinary machine models, one may require that $\mathcal{R}_n(A; z)$ is defined exactly when the computation halts and that bounded-resource predicates are decidable. The present note does not need this full recursion-theoretic generality, because the main examples are finite decision and oracle models. What we do use is the same discipline: there is no model-free quantity called "the time." The primitive is the resource functional $\mathcal{R}$, traditionally a complexity measure. For a fixed algorithm, the map $x \mapsto r^{\mathcal{R}}_{A,n}(x)$ is its pointwise cost function. After optimizing over procedures specialized to x, the map $x \mapsto C^{\star}_{\mathcal{R},n}(x)$ is the pointwise resource complexity.

For a globally correct algorithm A, its *pointwise worst-case resource complexity* is

$$r^{\mathcal{R}}_{A,n}(x) := \sup_{\omega \in \Omega_n(x)} \mathcal{R}_n(A; x, \omega), \qquad x \in \mathcal{X}_n.$$

If there is no hidden completion, this convention is recovered by setting $\Omega_n(x) = \{\perp\}$, so the cost function is simply the resource of A on the ordinary input x. Worst-case complexity collapses this function to $\sup_{x \in \mathcal{X}_n} r^{\mathcal{R}}_{A,n}(x)$. Pointwise complexity keeps the function.

The conditional benchmark at a public point allows the procedure to be designed after seeing the public instance, but not after seeing the hidden completion. Formally, let $\mathfrak{A}_n[x]$ be the class of decision procedures specialized to x that are correct for every $\omega \in \Omega_n(x)$. Define

$$C^{\star}_{\mathcal{R},n}(x) := \inf_{B \in \mathfrak{A}_n[x]} \sup_{\omega \in \Omega_n(x)} \mathcal{R}_n(B; x, \omega).$$

This is the clairvoyant benchmark used throughout the note. It knows the revealed instance x, including its representation if the model makes that representation part of the input. It does not know the hidden solution ω. A global algorithm A is pointwise optimal, or universally instance-optimal, for the resource $\mathcal{R}$ if there are universal constants c and an explicitly stated additive baseline $b_n(x)$ such that

$$r^{\mathcal{R}}_{A,n}(x) \le c\, C^{\star}_{\mathcal{R},n}(x) + b_n(x) \qquad \text{for every } x \in \mathcal{X}_n.$$

For pure comparison resources in this paper, $b_n(x) = 0$. For total-time statements, $b_n(x)$ may contain unavoidable input-reading, output-writing, or graph-scanning terms; these terms must be declared rather than hidden inside the notation.

Auxiliary structures are not primitive unless the model says so. A transitive closure, tree decomposition, active set, preconditioner, entropy vector, heap trace, or balanced comparison tree may be crucial in a proof or implementation. If it is supplied as part of x, then the benchmark may use it. If an algorithm constructs it, the construction cost belongs to $\mathcal{R}$. Thus pointwise complexity is compatible with representation issues, but it does not confuse an analytic witness for the public instance itself.

Example 2.2 (Partial-information sorting)**.** *In the sorting problem studied below, the public instance is a partial order $x = P$, the hidden completions are*

$$\Omega_n(P) = \mathrm{Lin}(P),$$

the correctness relation requires outputting the hidden extension, and $\mathcal{R}$ is the number of additional comparisons with the hidden total order. The conditional benchmark $C^{\star}_{\mathcal{R},n}(P)$ is the minimum height of a comparison tree built specifically for P. Theorem 4.4 shows that one uniform algorithm matches this benchmark up to a constant for every P.

Example 2.3 (Universal optimality for Dijkstra)**.** *The same definition also captures universal optimality for weighted graph algorithms. For the distance-ordering problem of Haeupler et al. (2024), the public instance is a directed or undirected graph topology together with a source, say $x = (G, s)$. The hidden completions are positive edge-length assignments $w : E(G) \to \mathbb{R}_{>0}$, with a fixed tie-breaking convention if needed. The correctness relation requires outputting the vertices in increasing distance from s. The pointwise cost of an algorithm is*

$$r_A^{\mathcal{R}}(G, s) = \sup_w \mathcal{R}(A; G, s, w),$$

where $\mathcal{R}$ may be running time in the comparison-addition model or the number of comparisons. The conditional optimum is the best algorithm designed for that fixed topology (G, s), still under worst-case edge lengths. In this notation, the universal optimality theorem for Dijkstra says that a single Dijkstra-type algorithm with a beyond-worst-case heap matches the topology-wise optimum, up to the resource conventions and unavoidable linear terms stated in that model. If randomized algorithms are included in the comparison class, the expectation is over internal randomness after (G, s, w) is fixed; no distribution over weights is required by the worst-case cost itself. This example is technically deeper than partial sorting, but it has the same quantifier structure: public topology, hidden weights, worst case over completions, and one global algorithm compared with a topology-wise clairvoyant benchmark.

More recent advances show that, for directed single-source shortest paths, the classical Dijkstra-style reliance on a globally sorted extraction order is not inherent: the sorting barrier can be broken by avoiding fully sorting all vertices within the dynamic-programming procedure (Duan et al., 2025).

Relation to existing instance-optimality and universal-optimality notions. The convention developed in this paper emphasizes the pointwise worst-case resource-complexity functional, and is meant to align with existing advanced notions of instance optimality and universal optimality rather than to claim a stronger replacement. In the order-oblivious geometric framework of Afshani et al. (2017), the completed input is an ordered sequence σ of points, while the base instance is the unordered point set $S = \rho(\sigma)$; the fibre $\Omega(S)$ consists of the permutations of S. Their guarantee compares one algorithm with the best algorithm on the worst, and in fact even the average, presentation order of the same point set. Thus their result is pointwise in the geometric instance after quotienting away input order. Universal optimality for Dijkstra-type graph algorithms has the same shape with a different fibre: the base instance is a graph topology (G, s), and the completions are adversarial edge weights.

Both Afshani et al. (2017) and Haeupler et al. (2024) explicitly use sorting as a motivating analogue for their frameworks. The authors of Haeupler et al. (2024) later resolved sorting under partial information as an illustration of related techniques, while reserving the term universal optimality primarily for graph algorithms (Haeupler et al., 2025). The present note emphasizes the underlying pointwise resource functional: the variable x is the public instance, while the worst case is taken over hidden completions ω. The resulting algorithmic guarantee may be described as universal; in this note, we use the more explicit phrase *pointwise universal optimality*. The associated pointwise worst-case resource complexity, however, is a resource functional of the public instance. In the partial-sorting example, the public instance is the partial order P, and the hidden completions are its linear extensions.

3 Sorting under Partial Information

Let $[n] = \{1, \dots, n\}$. A partial order P on $[n]$ is public information. A linear extension of P is a permutation π of $[n]$ such that whenever $i <_P j$, item i appears before item j in π. We write $\mathrm{Lin}(P)$ for the set of linear extensions and $e(P) = |\mathrm{Lin}(P)|$.

The hidden input is an unknown $\pi \in \mathrm{Lin}(P)$. An algorithm receives P, may adaptively query pairwise comparisons, and must output π. A query on (i, j) returns which of i and j appears first in the hidden extension. Comparisons already forced by P are public and need not be queried; the comparison cost counts only additional queries to the hidden total order.

For a deterministic algorithm A, let $q_A(P, \pi)$ be the number of additional comparisons made on hidden extension π, and define the public-point worst-case cost

$$Q_A(P) := \max_{\pi \in \mathrm{Lin}(P)} q_A(P, \pi).$$

The conditional optimum at P is

$$C^\star(P) := \inf_A Q_A(P),$$

where the infimum is over deterministic comparison decision procedures that are correct for every $\pi \in \mathrm{Lin}(P)$. Equivalently, $C^\star(P)$ is the minimum height of a comparison decision tree, allowed to be designed after P is fixed, that identifies every extension of P. The upper bounds below are stronger: they are achieved by uniform algorithms that receive P as input.

Definition 3.1 (Pointwise sorting dimension)**.** *The pointwise comparison dimension of a partial order P is*

$$d_{\mathrm{sort}}(P) := \log_2 e(P).$$

It is the number of binary comparison bits, in Shannon's logarithmic sense, that remain after the public information P has been revealed (Shannon, 1948).

4 A Bellman Potential for Universal Partial Sorting

We first record the elementary dynamic principle used in the proof. It is the finite decision analogue of a Bellman inequality.

Lemma 4.1 (Bellman potential)**.** *Consider an adaptive comparison problem whose public states are s, whose terminal states require no further query, and whose query q at state s leads to one of the successor states $s_{q,o}$. Suppose there is a nonnegative function Φ and a query choice $q(s)$ for every nonterminal state such that*

$$\Phi(s) \geq 1 + \max_o \Phi(s_{q(s),o}), \qquad \Phi(s) = 0 \text{ on terminal states.}$$

Then the algorithm that always asks $q(s)$ uses at most $\Phi(s_0)$ comparisons from the initial state s_0. If Φ is real-valued, the integer number of comparisons is at most $\lfloor \Phi(s_0) \rfloor$, and hence at most $\lceil \Phi(s_0) \rceil$.

Proof. Let $T(s)$ be the worst-case number of future comparisons made by the stated algorithm from state s. For terminal s, $T(s) = 0 \leq \Phi(s)$. For nonterminal s,

$$T(s) = 1 + \max_o T(s_{q(s),o}).$$

Induction on the number of remaining feasible hidden states gives $T(s_{q(s),o}) \leq \Phi(s_{q(s),o})$, and therefore

$$T(s) \leq 1 + \max_o \Phi(s_{q(s),o}) \leq \Phi(s).$$

Since $T(s)$ is an integer, the stated rounded bounds follow. □

For posets, the needed potential is logarithmic. If i and j are incomparable in P, write $P^{i<j}$ and $P^{j<i}$ for the transitive closures obtained by adding the corresponding relation. The two sets of extensions form a partition:

$$e(P^{i<j}) + e(P^{j<i}) = e(P).$$

Theorem 4.2 (Balanced-pair theorem, Kahn–Saks)**.** *There is a universal constant $\beta < 1$ such that every finite partial order P that is not total has an incomparable pair (i, j) satisfying*

$$e(P^{i<j}) \le \beta e(P), \qquad e(P^{j<i}) \le \beta e(P).$$

One may take $\beta = 8/11$ by the theorem of Kahn and Saks (1984).

The full 1/3–2/3 conjecture would allow $\beta = 2/3$, but the weaker constant is already enough for $O(\log e(P))$ comparisons. The resulting potential is explicit.

Proposition 4.3 (Logarithmic Bellman potential for partial sorting)**.** *Let $\beta = 8/11$ and set*

$$\kappa_{\mathrm{KS}} := \frac{1}{\log_2(1/\beta)} = \frac{1}{\log_2(11/8)}.$$

For every finite partial order P, define

$$\Phi_{\mathrm{KS}}(P) := \kappa_{\mathrm{KS}} \log_2 e(P).$$

At each nonterminal state, choose the lexicographically first incomparable pair, among all pairs, that satisfies the two inequalities in Theorem 4.2. Then Φ_{KS} satisfies the Bellman inequality in Lemma 4.1. Consequently this uniform comparison rule sorts every hidden extension of P using at most

$$\lceil \kappa_{\mathrm{KS}} \log_2 e(P) \rceil$$

additional comparisons.

Proof. If P is total, then $e(P) = 1$ and $\Phi_{\mathrm{KS}}(P) = 0$. Otherwise choose the lexicographically first pair (i, j) satisfying Theorem 4.2. For either outcome $P' \in \{P^{i<j}, P^{j<i}\}$,

$$\Phi_{\mathrm{KS}}(P') = \kappa_{\mathrm{KS}} \log_2 e(P') \le \kappa_{\mathrm{KS}} \log_2(\beta e(P)) = \Phi_{\mathrm{KS}}(P) + \kappa_{\mathrm{KS}} \log_2 \beta = \Phi_{\mathrm{KS}}(P) - 1.$$

Thus $\Phi_{\mathrm{KS}}(P) \ge 1 + \max\{\Phi_{\mathrm{KS}}(P^{i<j}), \Phi_{\mathrm{KS}}(P^{j<i})\}$, and Lemma 4.1 applies. □

This is a concrete Bellman log-potential dynamic program. The state is the current public poset, the action is the incomparable pair to query, and Φ_{KS} is a resource-to-go upper bound whose one-step decrease pays for the query. This is the precise sense in which the proof instantiates the Bellman-sufficient complexity viewpoint of Xu (2026b). The argument proves the comparison upper bound cleanly, but it should not be confused with an efficient implementation. Finding the balanced pair by exact counting may require hard preprocessing, since counting linear extensions is #P-complete (Brightwell and Winkler, 1991). Polynomial-time and near-linear-time sorting under partial information require the deeper algorithmic machinery developed in the cited literature. The point of Proposition 4.3 is narrower: at the level of comparison resources, the potential is explicit and its one-step decrease proves universal instance-optimality.

Theorem 4.4 (Pointwise universal optimality of partial-information sorting)**.** *For every finite partial order P,*

$$C^\star(P) \geq \lceil \log_2 e(P) \rceil.$$

Moreover, there exists a single deterministic comparison algorithm A_{bal}, valid for every finite partial order, such that

$$Q_{A_{\mathrm{bal}}}(P) \leq \lceil \kappa_{\mathrm{KS}} \log_2 e(P) \rceil$$

for every P, with zero comparisons when $e(P) = 1$. Hence

$$C^\star(P) = \Theta(\log e(P))$$

for all P with $e(P) \geq 2$, and $C^\star(P) = 0$ exactly when $e(P) = 1$.

If the public information is given as an acyclic directed graph $G = (V, E)$ whose reachability order is P_G, then known efficient implementations sort using $O(\log e(P_G))$ additional hidden-order comparisons. Total running time depends on the input and preprocessing convention. Under the standard DAG-input convention, topological heapsort runs in $O(|V| + |E| + \log e(P_G))$ time and $O(\log e(P_G))$ hidden-order comparisons (Haeupler et al., 2025). In the preprocessing formulation, the unified-bound-heap algorithm gives $O(|E|)$ preprocessing and then $O(\log e(P_G))$ sorting time and comparisons in that model (Rutschmann, 2026).

Proof. The lower bound is the decision-tree counting bound, applied after conditioning on P. Fix P. A deterministic comparison procedure induces a binary decision tree on the answers that are not already forced by P; branches inconsistent with P may be deleted. Each root-to-leaf path is a sequence of additional comparison answers. At a leaf, the procedure outputs one extension of P. If two distinct extensions $\pi, \pi' \in \mathrm{Lin}(P)$ reached the same leaf, the procedure would output the same order on both hidden inputs and therefore would be wrong on at least one of them. Thus the tree must have at least $e(P)$ leaves. A binary tree of height h has at most 2^h leaves, so $h \geq \lceil \log_2 e(P) \rceil$. Taking the minimum over all decision trees for this fixed P gives $C^\star(P) \geq \lceil \log_2 e(P) \rceil$.

The comparison upper bound is Proposition 4.3. If $e(P) = 1$, the public order has a unique linear extension, so that extension can be output without querying the hidden order. Combining the lower and upper bounds proves the pointwise equivalence. The efficient total-time statements are the cited algorithmic theorems. □

The theorem is stronger than ordinary worst-case optimality. Taking the supremum over all partial orders on n items gives the usual comparison-sorting bound. Indeed, for every partial order P, the set $\mathrm{Lin}(P)$ is a subset of all $n!$ permutations, so $e(P) \leq n!$; equality holds exactly in the no-information case, where no two distinct items are publicly comparable. Thus ordinary worst-case sorting is the maximum of the pointwise comparison complexity, while the same globally valid algorithm also pays only for the remaining extension entropy at every public point P.

Equivalently, Theorem 4.4 is an instance-optimality theorem with the instance chosen correctly. The instance is not the hidden total order π, because no correct algorithm may know π in advance. The instance is the revealed partial order P. Against the clairvoyant benchmark that knows P and designs the best comparison tree for $\mathrm{Lin}(P)$, the uniform algorithm loses only a constant factor. This is the precise sense in which pointwise worst-case complexity is compatible with the usual complexity-theoretic discipline of comparing to an instance-wise optimum without allowing the solution itself to be revealed.

Corollary 4.5 (A pointwise refinement of worst-case sorting)**.** *Let $\mathcal{P}_n$ be the family of partial orders on $[n]$. There is a globally valid deterministic comparison-sorting algorithm A_{bal} whose cost satisfies*

$$Q_{A_{\mathrm{bal}}}(P) = \Theta(d_{\mathrm{sort}}(P))$$

for every $P \in \mathcal{P}_n$ with $e(P) \geq 2$, while

$$\sup_{P \in \mathcal{P}_n} Q_{A_{\mathrm{bal}}}(P) = \Theta(n \log n).$$

Thus $P \mapsto \log e(P)$ is an exact pointwise refinement of the scalar worst-case sorting complexity.

5 Examples and Clarification

No information: the worst public point. An antichain is simply the partial order with no known comparison outcomes: no two distinct items are publicly comparable. Then every permutation is compatible with the public information, so $e(P) = n!$. Stirling's formula gives

$$d_{\mathrm{sort}}(P) = \log_2 n! = \Theta(n \log n).$$

Since no partial order can have more than $n!$ linear extensions, this no-information case is the public point that recovers the ordinary worst-case theorem for comparison sorting.

Complete information. If P is a chain, then $e(P) = 1$. The hidden order is already determined. The pointwise cost is zero additional comparisons, even though the input size is still n. If total running time rather than comparisons is measured, the cost of reading and outputting the public order must be charged.

Ordered blocks. Suppose the items are partitioned into blocks $B_1, \ldots, B_r$, the public information says that every item in B_a precedes every item in B_b whenever $a < b$, and there is no information inside a block. Then

$$e(P) = \prod_{a=1}^{r} |B_a|!, \qquad d_{\mathrm{sort}}(P) = \sum_{a=1}^{r} \log_2(|B_a|!).$$

The dimension adds across independent unresolved regions. A good algorithm should spend comparisons inside the blocks and none across blocks; the theorem says that, up to constants, a single global algorithm achieves exactly this behavior.

Merging two sorted lists. If P consists of two chains of lengths a and b, then sorting is merging. The number of compatible total orders is

$$e(P) = \binom{a+b}{a},$$

so

$$d_{\mathrm{sort}}(P) = \log_2 \binom{a+b}{a} = (a+b) H_2\left(\frac{a}{a+b}\right) + O(\log(a+b)),$$

where H_2 is the binary entropy function. The information-theoretic cost of merging is therefore another point on the same pointwise complexity curve.

Clarifying resources and instances The partial-information theorem is clean because every object is visible. The instance is the public point P. The remaining state space is $\mathrm{Lin}(P)$. The resource is additional comparisons. The dimension is the logarithm of the remaining state space. The theorem proves that this dimension is both necessary and sufficient for computation in the comparison model.

This clarifies the role of representations. A DAG representation, a transitive closure, a transitive reduction, a graph-entropy computation, a balanced decision tree, or a heap trace can be very important for proving or implementing the upper bound. But the comparison dimension $\log e(P)$ does not depend on which representation is used. If one changes the resource from comparisons to total running time, the representation immediately becomes part of the model: reading a sparse DAG, querying a partial-order oracle, and preprocessing a closure are different computational tasks.

The number $e(P)$ is not necessarily a quantity the algorithm must compute exactly; rather, $\log e(P)$ is the pointwise comparison complexity used to analyze the algorithm's behavior. Counting linear extensions is #P-complete (Brightwell and Winkler, 1991). Thus the significance of the upper bound is not that an efficient algorithm first evaluates $e(P)$, but that its comparison behavior can be analyzed by $\log e(P)$. Proposition 4.3 separates the comparison-theoretic Bellman idea from computationally efficient pair selection; the efficient algorithms cited in Theorem 4.4 address the latter issue.

The theorem also separates pointwise analysis from neighboring notions. Average-case analysis integrates the cost function over a distribution on public partial orders. Generic-case analysis asks for a bound on a large subset of public partial orders. Parameterized analysis upper-bounds the pointwise cost by width, number of incomparable pairs, poset dimension, or another chosen parameter. These are useful summaries. They are not the pointwise optimum itself. For sorting under partial information, the intrinsic pointwise comparison complexity is exactly $P \mapsto \log e(P)$, up to constants.

6 Finite-Resolution Sorting of Real Keys

Partial orders are finite. Numerical sorting is not: the keys may be real numbers. The pointwise idea remains meaningful once the revealed information is finite-resolution. This application is the sorting analogue of the finite-scale reduction used in pointwise statistical dimension (Li and Xu, 2026), while remaining a deterministic comparison theorem.

Let $x = (x_1, \dots, x_n) \in [0,1)^n$, with ties broken by item index. Fix an integer $b \geq 0$ and reveal the bucket labels

$$q_b(i) := \lfloor 2^b x_i \rfloor \in \{0, \dots, 2^b - 1\}.$$

These labels induce an ordered-block partial order $P_b(x)$: every item in a smaller bucket must precede every item in a larger bucket, while items in the same bucket remain unresolved. Let

$$B_a(x,b) := \{i : q_b(i) = a\}, \qquad m_a(x,b) := |B_a(x,b)|.$$

Then

$$e(P_b(x)) = \prod_{a=0}^{2^b-1} m_a(x,b)!.$$

Theorem 6.1 (Finite-resolution pointwise comparison complexity)**.** *After the b-bit bucket labels of the real keys are revealed, the optimal worst-case number of additional exact comparisons is*

$$C_b^\star(x) = \Theta\left(\sum_{a=0}^{2^b-1} \log_2(m_a(x,b)!)\right),$$

with the convention that the contribution of buckets of size 0 or 1 is zero. The upper bound is achieved by one uniform algorithm: bucket the items by their revealed labels and comparison-sort inside each nontrivial bucket.

Proof. The bucket labels impose a public ordered-block poset. A compatible exact total order is obtained by independently choosing the order inside each bucket, so the number of hidden completions is $\prod_a m_a!$. The decision-tree lower bound from Theorem 4.4 gives

$$C_b^\star(x) \geq \left\lceil \log_2 \prod_a m_a! \right\rceil = \left\lceil \sum_a \log_2(m_a!) \right\rceil.$$

For the upper bound, use merge sort inside every bucket and never compare elements from distinct buckets. Let $M(m)$ be the worst-case number of comparisons made by this fixed merge sort on m elements, with

$$M(0) = M(1) = 0, \qquad M(m) = M(\lfloor m/2 \rfloor) + M(\lceil m/2 \rceil) + m - 1 \quad (m \geq 2).$$

Then $M(m) \leq m\lceil \log_2 m \rceil$. Also $\log_2(m!) \geq (m/2)\log_2(m/2)$ for $m \geq 2$, after adjusting the absolute constant for $m = 2, 3$. Hence $M(m) \leq C \log_2(m!)$ for a universal constant C and all $m \geq 2$. Summing over buckets gives

$$\sum_a M(m_a) \leq C \sum_a \log_2(m_a!).$$

This proves the matching upper bound. □

The merge-sort upper bound also has an explicit Bellman potential. During a merge of two sorted lists of current lengths $r, s \geq 1$, take the potential to be $r + s - 1$. Comparing the two current first elements removes one element from one list, so the potential falls by exactly one in both outcomes. During the recursive sorting phase on a block of size m, take the potential to be $M(m)$; splitting the block into $\lfloor m/2 \rfloor$ and $\lceil m/2 \rceil$ creates two recursive subproblems and a future merge budget of $m - 1$, exactly matching the recurrence for $M(m)$. Summing this potential over all buckets gives a fully explicit Bellman proof of the upper bound in Theorem 6.1.

This theorem is the promised "beyond finite resolution" application inside sorting. With $b = 0$, all items lie in one bucket and the cost is $\log_2 n! = \Theta(n \log n)$. If b is large enough that every bucket contains at most one item, the revealed prefixes already determine the sorted order and no further comparisons are needed. At intermediate resolutions, the cost is exactly the entropy of the unresolved buckets, up to constants.

There is also a probabilistic reading. Suppose $X_1, \ldots, X_n$ are i.i.d. from a continuous distribution on $[0, 1)$. Conditional on the bucket labels $(q_b(i))_{i=1}^n$, the relative order of the items inside each bucket is uniform, by exchangeability, and the buckets are already ordered. Therefore the conditional probability of the realized exact order is

$$\prod_a \frac{1}{m_a!},$$

and the conditional code length of the exact rank order is precisely

$$-\log_2 \Pr\{\mathrm{ord}(X) = \mathrm{ord}(x) \mid q_b(X) = q_b(x)\} = \sum_a \log_2(m_a!).$$

Thus the comparison cost, the finite-resolution code length, and the residual bucket entropy coincide in this example.

7 Connections with Occam, Finite-Scale Dimension, and Polytopes

The logarithmic quantity should not be identified with only one neighboring theory. It has three typed readings. In a finite residual state space it is an Occam length. In finite-resolution numerical sorting it is a residual cell dimension, the discrete analogue of finite-scale pointwise statistical dimension. Through Stanley's order polytope it is also a polyhedral log-volume. These readings are useful precisely because they are kept separate. The Bellman potential is a fourth, algorithmic object: it is the dynamic-programming proof that the pointwise bound is attainable by one uniform comparison rule. We begin with the finite Occam specialization.

Let S be a finite nonempty set and let $\Delta(S)$ be the set of probability distributions on S. For $\mu \in \Delta(S)$, define the singleton pointwise code length

$$D_\mu(s) := \log_2 \frac{1}{\mu(s)}, \qquad s \in S,$$

with the convention $D_\mu(s) = +\infty$ if $\mu(s) = 0$. Then

$$\inf_{\mu \in \Delta(S)} \sup_{s \in S} D_\mu(s) = \log_2 |S|.$$

Indeed, for every μ, some point has mass at most $1/|S|$, while the uniform distribution attains equality. Applying this identity to $S = \mathrm{Lin}(P)$ gives

$$d_{\mathrm{sort}}(P) = \inf_{\mu \in \Delta(\mathrm{Lin}(P))} \sup_{\pi \in \mathrm{Lin}(P)} \log_2 \frac{1}{\mu(\pi)}.$$

Thus $\log e(P)$ is exactly the optimal worst-case Occam length of the finite set of hidden total orders still compatible with P (Blumer et al., 1987).

The same number has a polyhedral, and hence linear-programming, shadow. The partial order P defines Stanley's order polytope

$$\mathcal{O}(P) := \{x \in [0,1]^n : x_i \le x_j \text{ whenever } i \le_P j\}.$$

This is a feasible region cut out by linear inequalities. Its canonical triangulation has one simplex of volume $1/n!$ for each linear extension of P, and Stanley's formula gives

$$\mathrm{Vol}(\mathcal{O}(P)) = \frac{e(P)}{n!} \qquad \text{or equivalently} \qquad \log e(P) = \log n! + \log \mathrm{Vol}(\mathcal{O}(P)).$$

Thus $\log e(P)$ can also be read as the discrete log-volume of the remaining feasible ranking region (Stanley, 1986). This does not make partial-information sorting an algorithm for general linear programming. The sorting problem identifies a hidden ranking by adaptive comparison queries, whereas linear programming optimizes a linear objective over a polytope. But the analogy is useful:

revealed inequalities shrink a feasible region, and the pointwise complexity measures the remaining combinatorial or volumetric size of that region.

There is an equivalent vertex view through the linear-ordering polytope. Encode a total order by pairwise variables $y_{ij} \in \{0,1\}$, where $y_{ij} = 1$ means that i precedes j. The public constraints $i <_P j$ fix the corresponding coordinates to one, and the remaining feasible ranking vertices are precisely the linear extensions of P. A comparison query asks for one still-unfixed coordinate of the hidden vertex. In this form, partial-information sorting identifies a hidden vertex of a public face of a ranking polytope.

The relation to pointwise statistical dimension is obtained by replacing finite singletons by finite-resolution metric balls. If S is equipped with the discrete metric $\rho(s,t) = \mathbf{1}\{s \neq t\}$, then for every $\varepsilon < 1$, the ball $B_\rho(s,\varepsilon)$ is the singleton $\{s\}$, and the finite identity above becomes

$$\inf_{\mu \in \Delta(S)} \sup_{s \in S} \log_2 \frac{1}{\mu(B_\rho(s,\varepsilon))} = \log_2 |S|.$$

The pointwise statistical dimension framework of Li and Xu (2026) starts from the same prior-mass template but removes the finite/discrete restriction. There the hypothesis class $\mathcal{F}$ may be uncountable, the distance ρ is a statistical semimetric, and the pointwise quantity has the form

$$D_\Pi(f,\varepsilon) := \log \frac{1}{\Pi(B_\rho(f,\varepsilon))}, \qquad B_\rho(f,\varepsilon) = \{g \in \mathcal{F} : \rho(f,g) \leq \varepsilon\},$$

possibly with optimization or integration over scales. The ball is essential: under a continuous prior, $\Pi(\{f\}) = 0$ for typical f, so the singleton Occam length would be infinite. Finite-scale neighborhoods turn the question into: how much prior mass lies in a statistically indistinguishable neighborhood of this hypothesis? In smooth or representation-rich models, that mass is governed by local geometry, effective rank, and learned feature spectra rather than by finite cardinality.

Finite-resolution sorting is the exact bridge between the finite and uncountable pictures inside the sorting model. The real vector $x \in [0,1)^n$ is uncountable, but the revealed bucket cell

$$\{y \in [0,1)^n : q_b(y_i) = q_b(x_i) \text{ for all } i\}$$

induces a finite residual ordering problem. The pointwise complexity is not the code length of the singleton x, which would be infinite under continuous priors. It is the log-size of the finite set of rank orders still possible at resolution 2^{-b}:

$$\sum_a \log_2(m_a!).$$

Thus the citation to Li–Xu is right in a typed sense: their pointwise dimension motivates the finite-scale move from singleton states to neighborhoods or cells in uncountable spaces, while the theorem proved here remains a deterministic comparison-complexity theorem.

The mapping is therefore exact but typed:

finite sorting state space	$\mathrm{Lin}(P)$,
finite Occam envelope	$\log \lvert\mathrm{Lin}(P)\rvert$,
finite-resolution real-key cell	$\prod_a m_a!$ residual rankings,
polyhedral shadow	$\log n! + \log \mathrm{Vol}(\mathcal{O}(P))$,
discrete metric ball at $\varepsilon < 1$	$\{\pi\}$,
statistical analogue	$\log 1/\Pi(B_\rho(f,\varepsilon))$,
resource controlled here	additional comparisons,
resource controlled in learning	generalization or estimation error.

The shared principle is localization before taking the worst case. The mathematical objects and units are different. The present theorem is not a statistical generalization bound; it is a comparison-decision theorem. Conversely, the pointwise dimension of Li and Xu (2026); Lutz (2016) is not a sorting complexity; it is a finite-scale statistical complexity for hypothesis classes that may be uncountable.

The same distinction clarifies the relation with Bellman-sufficient information and computational complexity (Xu, 2026b). A code length can be assigned to the realized extension π conditional on P, and some extensions may have very short descriptions. Sorting under partial information asks for a minimax guarantee: one fixed comparison algorithm must be correct for every $\pi \in \mathrm{Lin}(P)$, and the pointwise complexity takes the worst case over this compatible set. The operational length is therefore the minimax code length of the remaining set, $\log|\mathrm{Lin}(P)|$, not the description length of a lucky realized extension. The Bellman potential in Section 4 is the computational counterpart: it is a resource-to-go function whose one-step decrease proves a uniform worst-case upper bound, and in this note the resource-to-go function is the explicit log potential Φ_{KS}.

There is also a high-level connection with information-relaxation formulations of algorithmic robustness. For example, Xu (2026a) uses minimax language to compare what becomes possible when additional information is revealed to an algorithmic procedure. The present note studies the offline computational analogue in its cleanest finite form: reveal P, take the worst case over $\mathrm{Lin}(P)$, and compare a global algorithm with the benchmark that is allowed to know P. This shared quantifier pattern should not be confused with a shared formal framework. Here the resource is additional comparisons; in statistical or robustness problems the resource may be risk, loss, sample size, or an algorithm-dependent performance gap.

Universal graph algorithms give a deterministic-computation example of the same template rather than merely an analogy. In Haeupler et al. (2024), the public object is a graph topology, the hidden completions are edge weights, and the pointwise resource cost is a worst case over weights at that fixed topology. The comparison with a topology-wise optimum is exactly the pointwise worst-case comparison in Section 2, specialized to graph algorithms and their chosen resource model. The sorting theorem remains useful because its hidden-completion space and logarithmic quantity can be written down without the additional data-structure machinery required by Dijkstra.

Thus pointwise does not mean nonuniform, and localized does not always mean geometric. In finite comparison problems, localization may reduce to an Occam-like finite state count. In finite-resolution numerical problems, it becomes the size of a residual cell. In uncountable statistical problems, localization requires metric balls at a chosen scale. In polyhedral problems, localization may appear as the volume or vertex structure of a feasible region. In all cases, the discipline is the same: fix the global rules first, and only then keep the pointwise cost function instead of collapsing it immediately to its maximum.

8 Outlook

Sorting with partial information is a solved benchmark for pointwise worst-case complexity. The residual dimension $\log e(P)$ is explicit, the lower bound is elementary, and a logarithmic Bellman potential gives a transparent comparison upper bound. The finite-resolution real-key variant shows how the same pointwise logic moves from finite residual sets to uncountable inputs by working at a chosen scale. Efficient algorithms add a separate layer: they show that the favorable comparison bound can be attained without spending prohibitive time to discover the right comparisons.

The graph-algorithm example shows that this is not only a sorting phenomenon. Universal optimality of Dijkstra refines worst-case shortest-path complexity by fixing the graph topology

and taking the worst case only over edge weights (Haeupler et al., 2024). What is special about sorting under partial information is that the conditional state space is finite and explicit: $\mathrm{Lin}(P)$. In shortest paths, optimization, and learning, the same pointwise discipline remains valuable, but the pointwise complexity may depend on data structures, numerical conditioning, communication model, finite-scale geometry, or the cost of discovering favorable structure.

The moral is deliberately narrow and useful. First decide what information is legitimately revealed, and make that information the public instance. Then compare one global algorithm with the clairvoyant benchmark that knows this instance but not the hidden completion. When the global cost function matches the benchmark point by point, worst-case complexity has a genuine pointwise refinement, and universal instance-optimality is achieved without abandoning uniform computation. Partial-information sorting shows this principle in finite form; finite-resolution sorting shows how the same principle begins to operate on uncountable numerical inputs.

Acknowledgements

This note is primarily a reformulation of the modern theory of sorting under partial information. It identifies that theory as an exact example of pointwise worst-case computational complexity; it views the log-potential argument as a deterministic-comparison instantiation of the Bellman-sufficient complexity viewpoint; and it adds finite-resolution sorting of real keys as a simple application. These statements are meant in the deterministic comparison model, not in a statistical or learning sense.

The author used ChatGPT 5.5 Pro as a research and editorial assistant for generating technical suggestions and improving the clarity of presentation. Definition 2.1 were developed with assistance from ChatGPT 5.5 Pro. The author assumes full responsibility for the final formulation, correctness, and validity of the work.

References

Peyman Afshani, Jérémy Barbay, and Timothy M. Chan. Instance-optimal geometric algorithms. *Journal of the ACM*, 64(1), Article 3, 2017.

Manuel Blum. A machine-independent theory of the complexity of recursive functions. *Journal of the ACM*, 14(2):322–336, 1967.

Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, and Manfred K. Warmuth. Occam's razor. *Information Processing Letters*, 24(6):377–380, 1987.

Graham Brightwell and Peter Winkler. Counting linear extensions. *Order*, 8:225–247, 1991.

Jean Cardinal, Samuel Fiorini, Gwenael Joret, Raphael M. Jungers, and J. Ian Munro. Sorting under partial information (without the ellipsoid algorithm). *Combinatorica*, 33:655–697, 2013.

Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, and Longhui Yin. Breaking the sorting barrier for directed single-source shortest paths. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC 2025)*, pages 36–44, 2025.

Michael L. Fredman. How good is the information theory bound in sorting? *Theoretical Computer Science*, 1(4):355–361, 1976.

Bernhard Haeupler, Richard Hladik, John Iacono, Vaclav Rozhon, Robert E. Tarjan, and Jakub Tetek. Fast and simple sorting using partial information. In *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 3953–3973. SIAM, 2025.

Bernhard Haeupler, Richard Hladik, Vaclav Rozhon, Robert E. Tarjan, and Jakub Tetek. Universal optimality of Dijkstra via beyond-worst-case heaps. In *Proceedings of the 65th IEEE Symposium on Foundations of Computer Science (FOCS)*, 2024.

Jeff Kahn and Jeong Han Kim. Entropy and sorting. *Journal of Computer and System Sciences*, 51(3):390–399, 1995.

Jeff Kahn and Michael E. Saks. Balancing poset extensions. *Order*, 1:113–126, 1984.

Shaojie Li and Yunbei Xu. Pointwise generalization in deep neural networks. arXiv:2605.18598, 2026.

Neil Lutz. A note on pointwise dimensions. arXiv:1612.05849, 2016.

Daniel Rutschmann. Sorting under partial information with optimal preprocessing time via unified bound heaps. arXiv:2604.12653, 2026.

Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423 and 27(4):623–656, 1948.

Richard P. Stanley. Two poset polytopes. *Discrete & Computational Geometry*, 1:9–23, 1986.

Ivor van der Hoog and Daniel Rutschmann. Tight bounds for sorting under partial information. In *Proceedings of the 65th IEEE Symposium on Foundations of Computer Science (FOCS)*, 2024.

Ivor van der Hoog, Eva Rotenberg, and Daniel Rutschmann. Simpler optimal sorting from a directed acyclic graph. In *Proceedings of the 2025 Symposium on Simplicity in Algorithms (SOSA)*, 2025.

Yunbei Xu. Pointwise complexity for Gaussian fields: Upper envelopes, algorithmic lower bounds, and separation. arXiv:2606.07931, 2026.

Yunbei Xu. Bellman-sufficient Information Complexity. arXiv:2606.11171, 2026.